

SemsAi

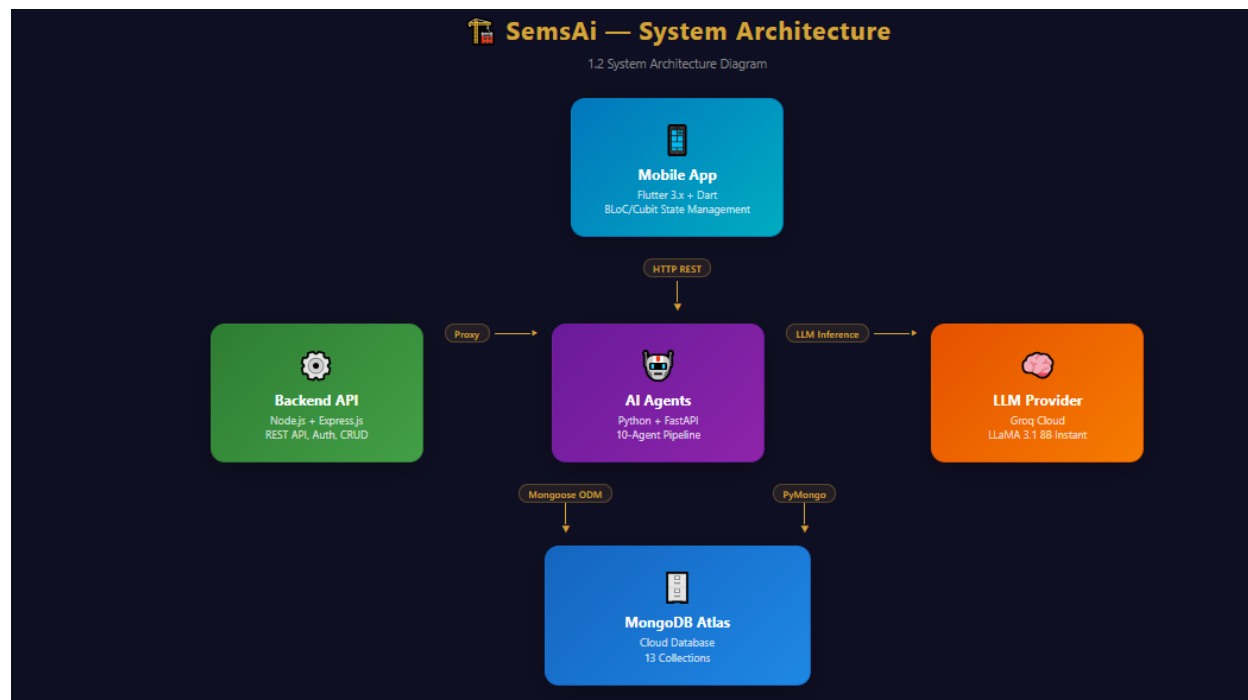
AI-Powered Real Estate Decision Support
System

2nd Seminar

IS Department — Graduation Projects
2025–2026

The project includes a full property browsing experience with an interactive map, property listings with advanced filters, and a favorites system.

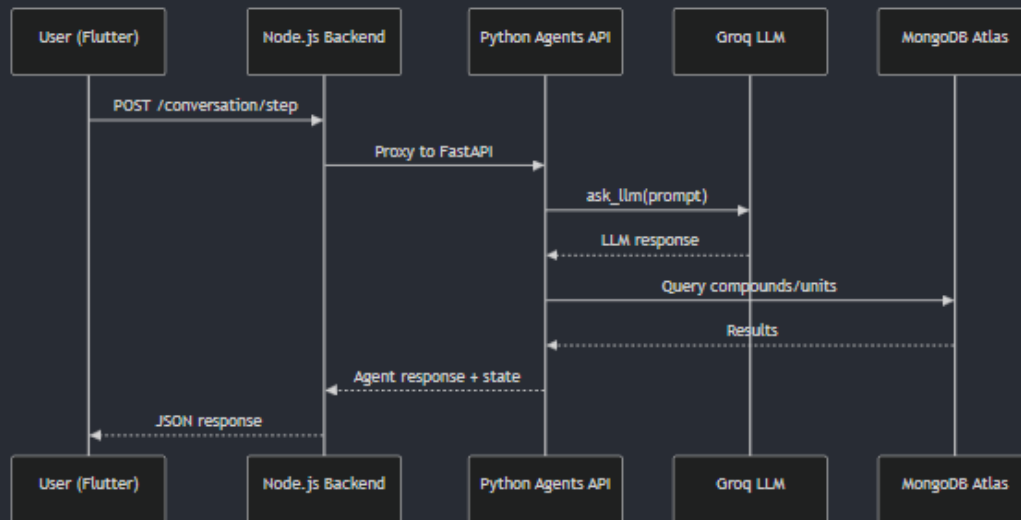
1.1 System Architecture



The system consists of the following layers:

Layer	Technology	Role
Mobile App	Flutter 3.x + Dart	Cross-platform UI with BLoC/Cubit state management
Backend API	Node.js + Express.js	REST API, Auth, CRUD, MongoDB queries, Agent proxy
AI Agents	Python + FastAPI	10-agent conversational AI pipeline
LLM	Groq Cloud (LLaMA 3.1 8B)	Natural language understanding & generation
Database	MongoDB Atlas	Stores compounds, units, developers, users, features

Data Flow for AI Conversation



2. List of Functions — Implemented vs. Not Yet Finished

2.1 Implemented Functions

Flutter Side:

#	Feature	Screen	Description
1	User Authentication	Login / Register / Forgot Password	Full auth flow with premium glassmorphism UI, email/password validation
2	Splash Screen	Splash	Animated app intro with branding
3	Bottom Navigation	Home	5-tab navigation: Listings, Explore, AI Chat, Favorites, Profile
4	Property Listings	Listings	Paginated grid with infinite scroll, search, sort, filter by region/type/bedrooms/price
5	Interactive Map	Explore	OpenStreetMap with compound markers, area chips, color coding, compound detail sheet
6	AI Chat	Agent Chat	Real-time conversation with AI assistant, typing indicator, phase labels
7	AI Recommendations	Recommendations	Top-3-ranked compounds with scores, reasons, unit details, animated entrance
8	Favorites	Favorites	Save/remove units locally, filter & sort favorites, search within favorites
9	Unit Detail	Unit Detail	Full unit info: image gallery, specs, payment plans, compound info
10	Profile	Profile	User info, default region selection (persisted), sign-out with confirmation
11	Compound Detail	Explore (Sheet)	Bottom sheet showing compound info, units list, view on map
12	Default Region	Profile → Listings/Explore	Setting a default region auto-filters listings and map

Backend (Node.js + Express):

#	API Endpoint	Method	Description
1	/auth/register	POST	User registration with duplicate email check
2	/auth/login	POST	Email/password authentication
3	/auth/forgot-password	POST	Password reset
4	/users	GET/POST	User CRUD operations
5	/users/locations	GET/POST/DELETE	Saved locations management
6	/developers	GET	List all developers
7	/compounds/map	GET	All compounds with lat/lng for map markers
8	/compounds/:id	GET	Single compound with full details + unit count
9	/compounds/:id/units	GET	Units belonging to a compound (limit 50)
10	/units/listings	GET	Paginated listings with filters (region, type, bedrooms, price, search, sort)
11	/units/filters	GET	Distinct regions and property types for filter UI
12	/units/:id	GET	Single unit detail with compound info
13	/explore/areas	GET	Area aggregation (top 15 areas by compound count)
14	/recommendations	POST	MongoDB aggregation pipeline for recommendations
15	/conversation/step	POST	Proxy to Python agents service

2.2 Not Yet Implemented Functions

#	Feature	Description	Priority
1	Developer Profile Pages	Dedicated page for each developer: profile, years active, projects, rating, reviews (if there), class, compounds	High
2	Property Comparison	Compare 2–3 properties side by side: price, area, amenities, location, developer, payment	High
3	Developer Contact	Communication methods per developer (phone, email, WhatsApp, website), direct contact from app (in case collab)	High
4	Map Clustering	Cluster nearby markers when zoomed out for better performance	Medium
5	Push Notifications	Alert users about new listings or price drops (in case of collab with developers)	Medium
6	Inflation Calculator	Financial tool to estimate property value appreciation	Medium
7	Search History	Save and recall previous AI chat conversations	Medium
8	Social Authentication	Login via Google/Facebook/Apple accounts	Low
9	Price Trend Charts	Historical price visualization per compound/area	Medium

3. Algorithms and Techniques

3.1 State Management — BLoC/Cubit Pattern (Flutter)

Technique: Cubit (simplified BLoC without events) for state management.

Cubits used:

- AuthCubit — Auth state (logged in/out, user data)
- HomeCubit — Home screen state
- FavoriteCubit — Local favorites management (SharedPreferences)
- ChatCubit — AI chat session management (session ID, messages, phase tracking)
- MapCubit — Map state

Justification:

- Cubit is lighter than full BLoC — no need for event classes for this project's complexity
- Separation of concerns — business logic separated from UI
- Reactive UI — UI rebuilds automatically when state changes
- Testable — business logic can be unit tested independently

3.2 MongoDB Aggregation Pipeline

Technique: Multi-stage MongoDB aggregation for complex queries.

Pipeline stages:

- \$lookup — Join units → payments
- \$match — Filter by payment type and budget
- \$lookup — Join → compounds
- \$match — Filter by location
- \$lookup — Join → developers

Justification:

- Server-side aggregation is far more efficient than fetching all data and filtering in application code
- \$lookup (joins) allow querying across collections in a single operation
- MongoDB Atlas supports this natively without additional infrastructure